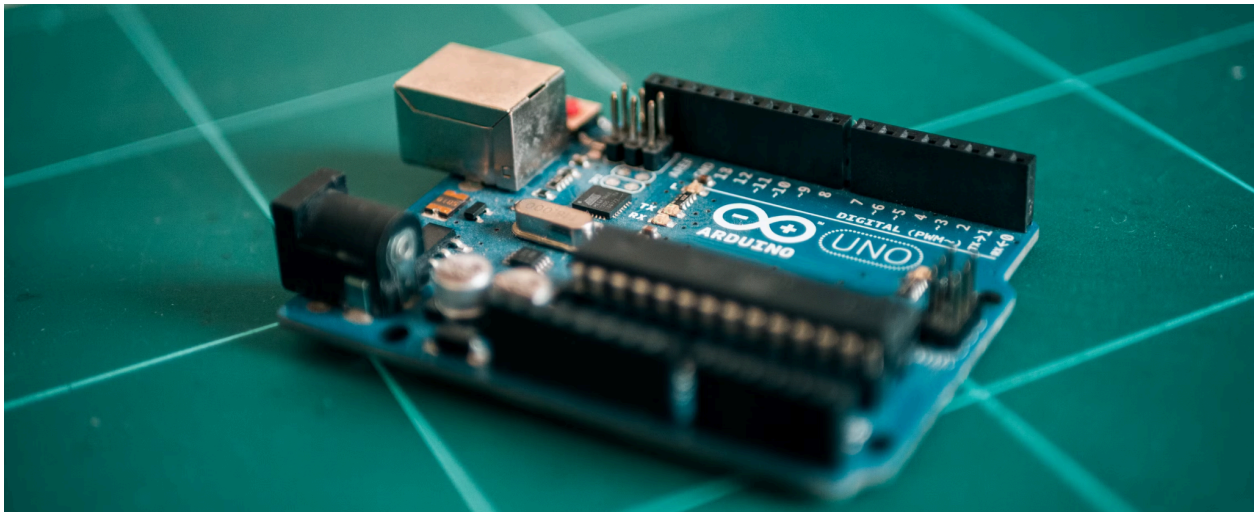
alessandro.virive@studenti.unicam.it link https://wokwi.com/projects/467806361810704385



Alessandro Virivè,

MAT-130052 UNICAM

CENTRALINA CONTROLLO ACCESSI

Architettura degli Elaboratori

Prof. Massimo Trojani

23-07-2026

Appello Parziale Modulo Lab AdE



Indice

I. Analisi **pag.2**

1.1 - Testo Assegnato pag.2

1.2 - Analisi del Problema (Flowchart Generale) pag.3

1.2.1 Analisi dei Casi Estremi

1.2.2 Vincoli imposti da componenti fisici

1.2.3 Isolamento dei ME

1.3 - Analisi della Soluzione Adottata pag.7

1.3.1 Statechart

1.3.2 Analisi del Flowchart

II. Sintesi **pag.13**

2.1 - Modulo Interrupt pag.13

2.2 - Modulo Generatore OTP pag.14

2.3 - Modulo Schedulatore Temporale pag.15

2.4 - Modulo Visivo pag.16

2.5 - Modulo Tastiera pag.18

III. Sistema Complessivo **pag.19**

3.1 Manuale di Utilizzo pag.19

I. Analisi

1.1 Testo assegnato

“Realizzare un sistema di controllo accessi costituito dalle seguenti periferiche:

a) due display a 7 segmenti da 4 + 2 cifre ciascuno del tipo: link componente

b) un keypad a 3 colonne e 4 righe {"attrs": {"columns": "3" }

c) un rilevatore di movimento: link componente

Il sistema quando rileva un movimento si pone in modalità operativa ovvero:

1) Premendo il tasto "" il sistema aspetta il codice personale da 6 cifre*

2) Riconosciuto il codice personale come autorizzato, invia un SMS con un codice OTP che dovrà essere inserito dal soggetto, preceduto dal tasto "#"

3) Se il codice OTP inserito è corretto e viene inserito entro 10 secondi, il sistema attiva l'apertura della cassaforte, altrimenti attiva il sistema di allarme

Deve essere possibile interrompere il processo di accesso digitando in sequenza "" e "#"*

In questa versione, il codice OTP verrà inviato e visualizzato sulla console seriale.

La progettazione dovrà essere svolta secondo il paradigma:

•Minori risorse impiegate per la macchina

•Maggior sforzo per la progettazione e sviluppo.

In particolare:

a) I componenti HW-SW dovranno essere "Automi" indipendenti che comunicano con variabili globali

b) Le cadenze temporali (ritardi ecc.) dovranno essere governati dall'ISR del clock

c) La progettazione delle ISR dovrà garantire tempi di servizio compatibili con le risposte al tempo reale del sistema.

d) La gestione degli I/O deve essere diretta, senza usare le funzioni di "Arduino" o librerie esterne.

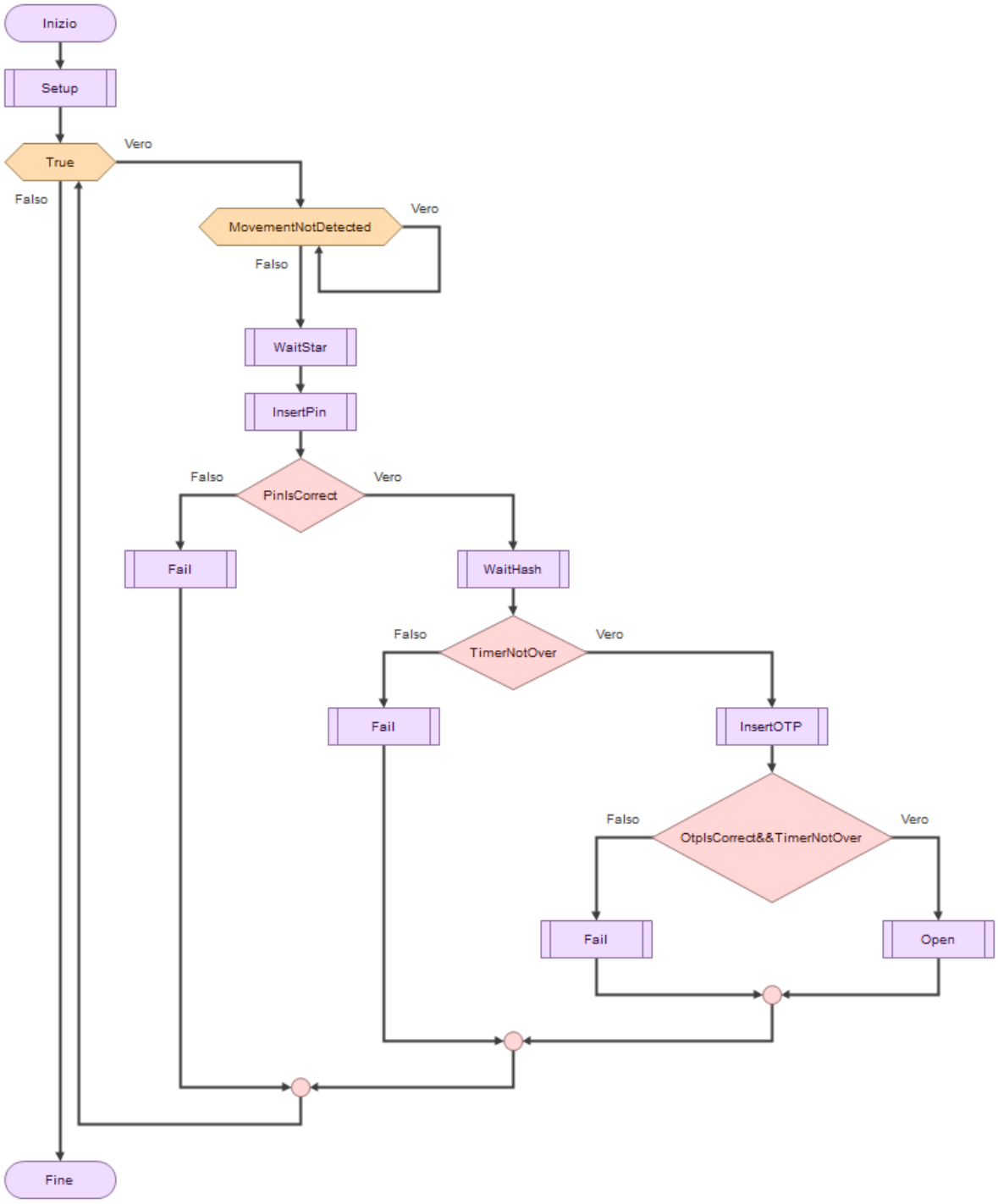
Ad esempio:

- Il keypad può essere gestito in foreground su loop ma senza usare funzioni di libreria.

- Le funzioni di visualizzazione sui display dovranno essere indipendenti e temporizzate tramite Interrupt del clock”

1.2 Analisi del Problema

Flowchart Generale



Il sistema è definito come una **Macchina a Stati Finiti** (FSM) reattiva agli eventi, composta da **moduli elaborativi indipendenti** comunicanti tramite variabili globali condivise.

Il sistema si trova in uno stato di riposo a basso consumo (Standby) finché non riceve un segnale di innesco tramite movimenti rilevato dal modulo PIR.

Ricevuto l'innesco, transita in uno stato di attesa attiva, viene abilitato il tastierino numerico (4 righe, 3 colonne) e il **display a 7 segmenti** a 6 cifre.

L'acquisizione del carattere di inizio stringa * avvia la **memorizzazione** di una sequenza numerica di 6 cifre (PIN). Durante questo processo il Pin dovrà essere mostrato sul display.

La comparazione positiva tra la sequenza inserita e una sequenza costante cablata genera una **sequenza numerica pseudo-casuale** di 6 cifre (OTP), **trasmessa su bus seriale**, e avvia un **conteggio temporale** di 10 secondi.

L'acquisizione del carattere di inizio stringa # avvia la **memorizzazione** di una seconda sequenza numerica.

La comparazione positiva tra la seconda sequenza e la sequenza OTP generata, valutata in AND logico con la validità del conteggio temporale (tempo < 10s), attiva il segnale di sblocco. Il fallimento di una comparazione o lo scadere del timer attivano lo stato di allarme.

L'acquisizione della sequenza *# sovrascrive lo stato corrente riportando il sistema allo stato di Standby, ad eccezione degli stati terminali (Allarme o Sblocco). Le operazioni di visualizzazione sono scorrelate dal flusso logico principale e governate da un clock asincrono.

1.2.1 Analisi dei casi estremi

1. **Digitazione in eccesso (Overflow del buffer):** Qualora l'utente continui a premere i tasti numerici dopo aver già raggiunto la lunghezza massima di 6 cifre per il PIN o per l'OTP, la condizione di acquisizione viene inibita. Il sistema scarta gli input in eccesso e mantiene inalterata la sequenza memorizzata fino alla sua valutazione.
2. **Comando di reset manuale:** L'acquisizione in sequenza del carattere asterisco seguito dal carattere cancelletto sovrascrive il normale stato di avanzamento. Se questa combinazione viene rilevata, il sistema forza l'azzeramento della memoria e il ritorno allo stato di attesa innesco. Tale regola viene valutata in AND logico con lo stato corrente per essere ignorata qualora il sistema si trovi già in uno stato terminale, come il blocco di sicurezza o la cassaforte aperta.
3. **Scadenza del tempo coincidente con l'inserimento:** Se la digitazione dell'ultima cifra dell'OTP avviene in concomitanza esatta con lo scadere dei 10 secondi massimi, il test temporale assume priorità gerarchica. Il sistema valuta prima la condizione di tempo scaduto, ignorando la correttezza del codice e attivando immediatamente l'allarme.

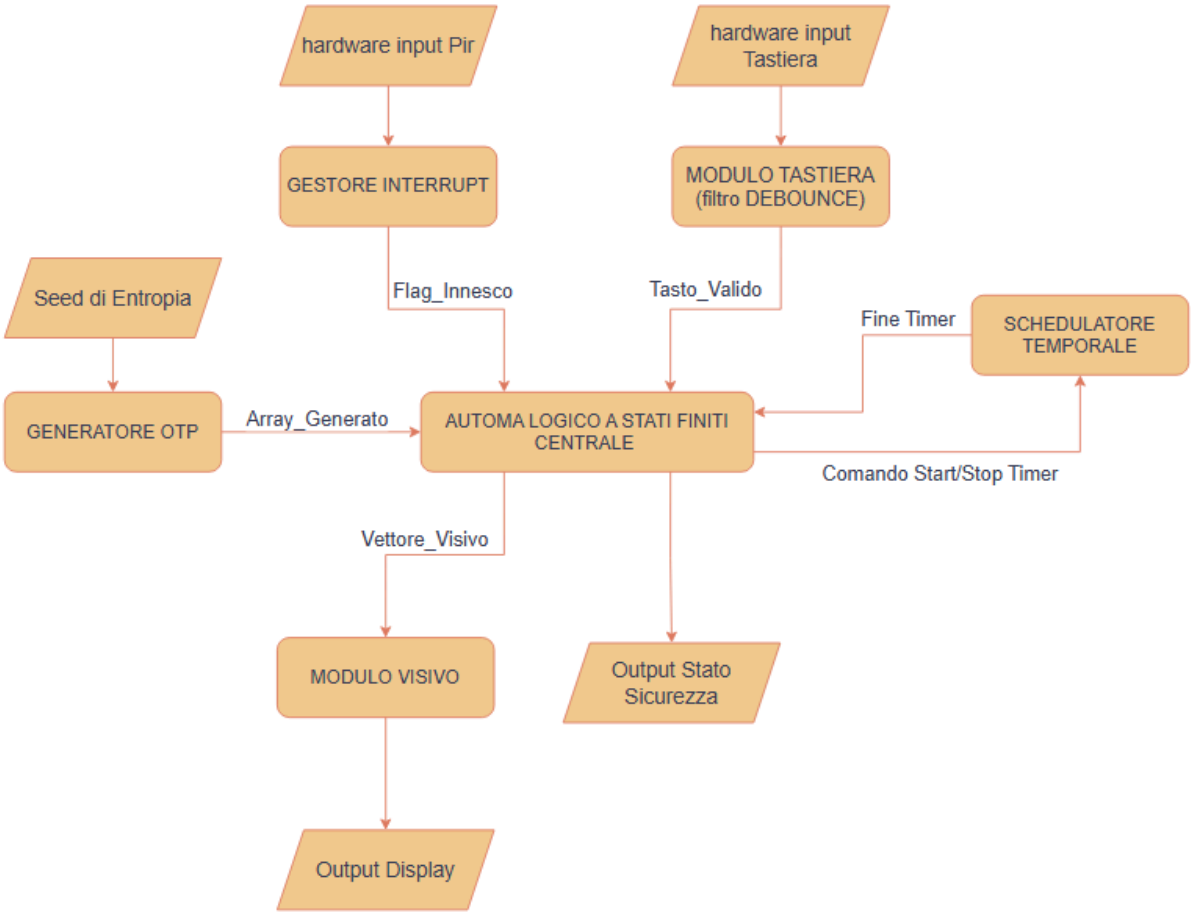
1.2.2 Vincoli imposti da componenti fisici

a) Vincolo di instabilità meccanica: I contatti fisici di un tastierino a matrice generano rimbalzi elettrici spuri al momento della pressione.
Modellizzazione: Interposizione di una **funzione logica di filtro antirimbalzo** (debounce) che attenui le transizioni fittizie. L'evento logico di pressione viene asseverato e validato solo se il valore dell'ingresso si mantiene stabile e identico per un numero costante "N" di cicli consecutivi del segnale di clock di sistema.
b) Vincolo di gestione display: La persistenza visiva richiede un aggiornamento ciclico e sequenziale per ciascuna delle 6 cifre, al fine di ottimizzare il numero di pin necessari ed evitare assorbimenti di corrente eccessivi.
Modellizzazione: Svincolare l'aggiornamento visivo dal flusso logico principale. L'operazione di lettura del vettore di stato visivo viene delegata a un **contatore asincrono indipendente che gestisce l'aggiornamento ciclico**.
c) Vincolo di assenza hardware entropico: La generazione pseudo-casuale del codice OTP necessita di un "seme" numerico di partenza imprevedibile, non potendo contare su un chip crittografico dedicato.
Modellizzazione: Utilizzo di un **contatore numerico ad incremento incondizionato** e ad altissima frequenza. L'evento di acquisizione dell'ultimo carattere del PIN (fortemente asincrono in quanto legato al tempo di reazione umana) blocca la lettura del contatore: il valore numerico in quell'esatto istante di campionamento fornisce la base entropica per il generatore.

1.2.3 Isolamento dei Moduli di Elaborazione

Per garantire l'assenza di blocchi e rispettare le specifiche di progettazione ad automi indipendenti, il sistema è disaccoppiato in Moduli di Elaborazione (ME) autonomi. Ogni modulo esegue test logici sulle proprie variabili di ingresso, applica una computazione specifica e aggiorna le variabili di uscita memorizzate nello spazio globale condiviso, agendo in parallelo agli altri processi.

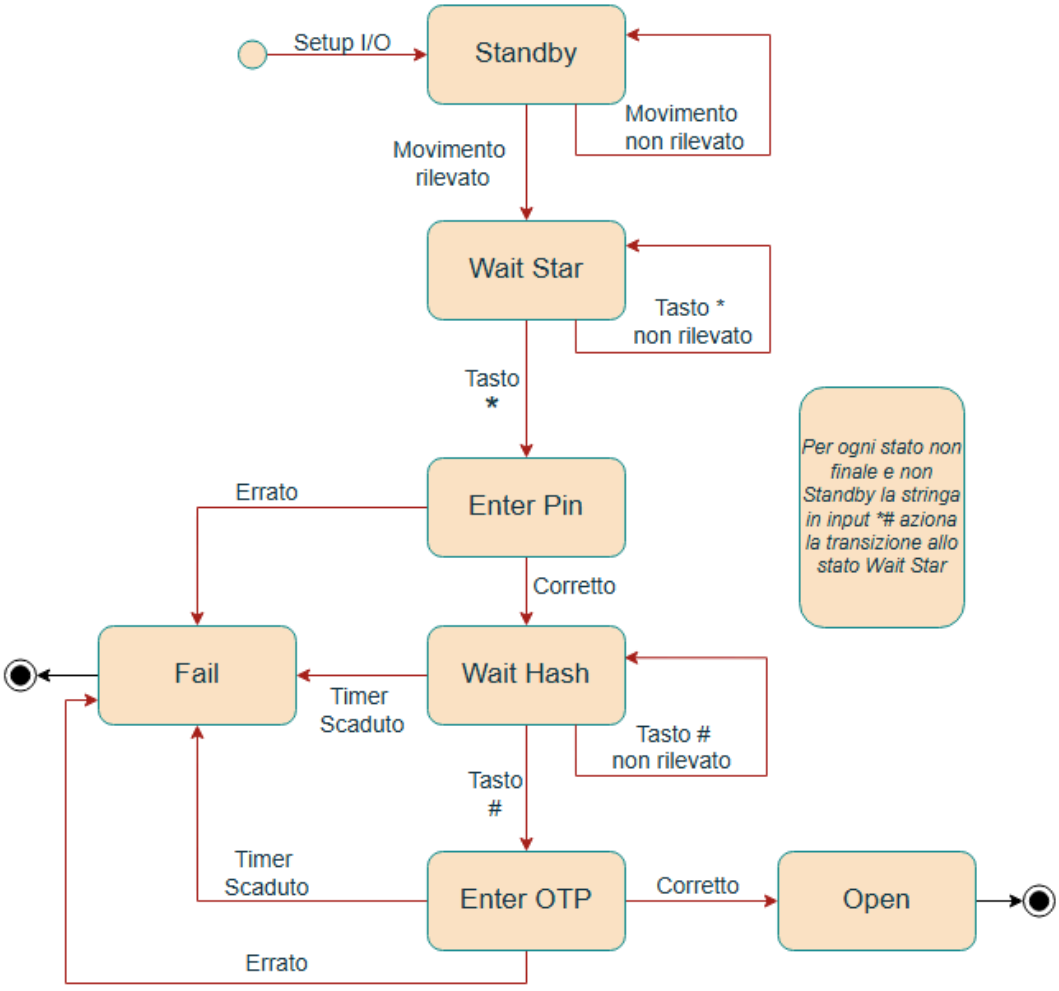
Modello di interazione tra Moduli di elaborazione:



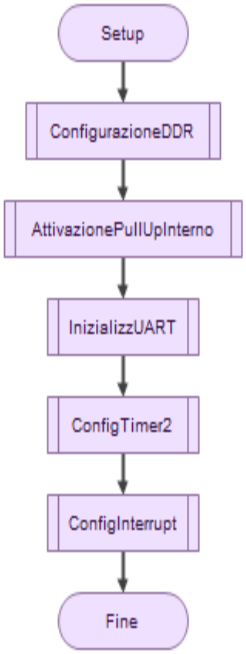
1.3 Analisi della Soluzione Adottata

1.3.1 Statechart

Uno STATECHART è un diagramma che visualizza lo stato di un sistema e le transizioni tra di essi. È composto da stati, eventi e azioni, offrendo una panoramica chiara del comportamento del sistema. È uno strumento essenziale per progettare e analizzare sistemi complessi.



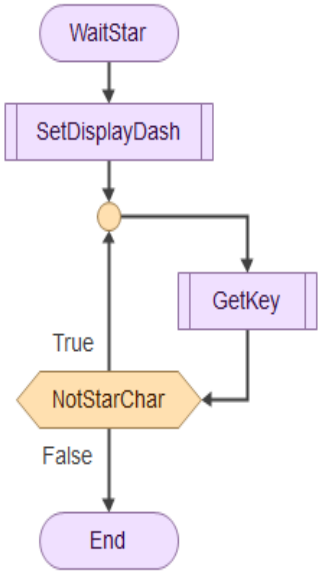
SETUP



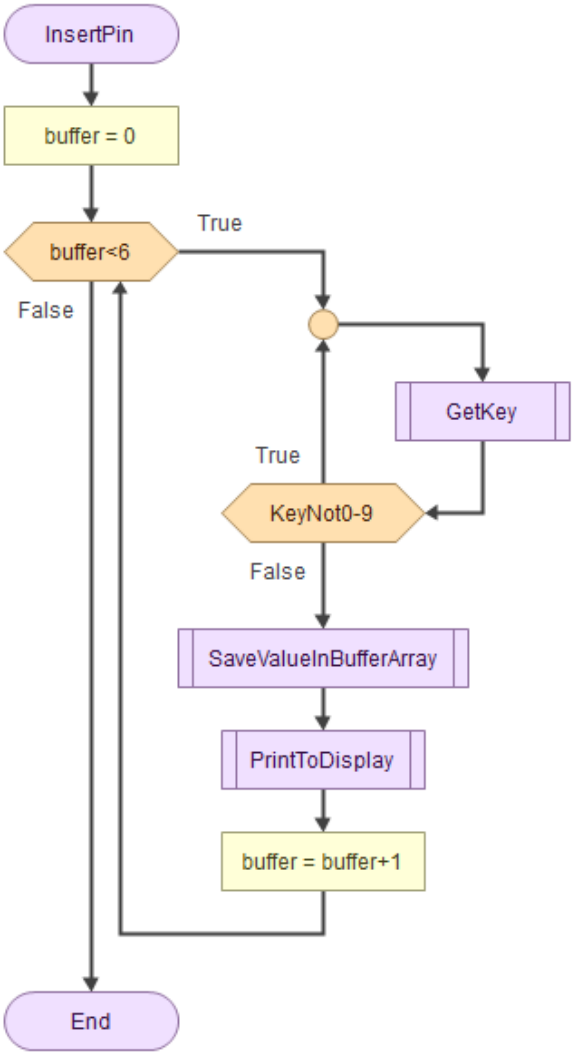
Il diagramma illustra la sequenza di inizializzazione (Boot/Setup) eseguita dal microcontrollore immediatamente dopo l'avvio o un Reset hardware. La fase di setup non si appoggia a librerie ad alto livello, ma prevede la scrittura sequenziale e diretta dei registri fisici della MCU. Il flusso logico è rigorosamente lineare e procede per blocchi funzionali.

WAIT STAR

Il diagramma illustra la fase di attesa attiva in cui il sistema transita immediatamente dopo essere stato risvegliato dal sensore volumetrico PIR. L'obiettivo di questo sottoprogramma è fornire un feedback visivo all'utente e intercettare l'esatta sequenza di innesco (la pressione del tasto *), bloccando l'esecuzione di ulteriori logiche fino a quando tale condizione non viene soddisfatta. Il flusso è modellato secondo un costrutto iterativo post-condizionale (simile a un ciclo *Do-While*)



INSERT PIN

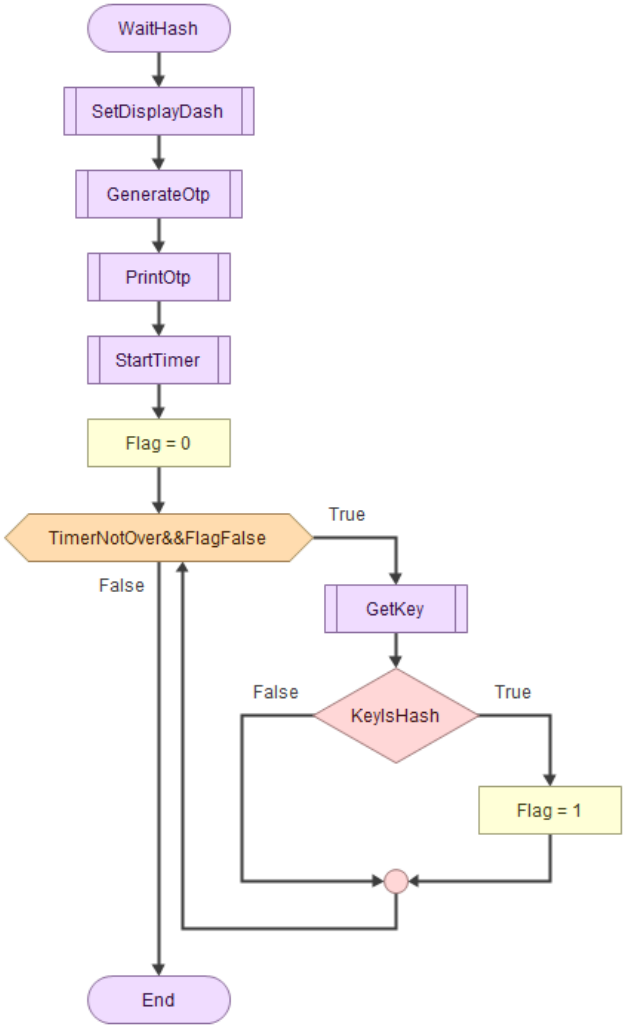


Il diagramma illustra l'acquisizione e validazione di un PIN a 6 cifre tramite una logica a cicli annidati:

- **Inizializzazione:** Il contatore delle cifre e l'indice di memoria vengono azzerati (`buffer = 0`).
- **Ciclo Principale (While):** Il sistema itera le operazioni finché il numero di cifre acquisite è strettamente inferiore a sei (`buffer < 6`).
- **Filtro Input (Do-While):** Un ciclo interno legge continuamente la tastiera (`GetKey`) e scarta l'input finché non viene rilevato un numero valido compreso tra 0 e 9, ignorando caratteri speciali.
- **Salvataggio e Feedback:** La cifra validata viene trascritta nell'array di memoria (`SaveValueInBufferArray`) e contestualmente inviata al display a 7 segmenti (`SendNumberToDisplay`).
- **Avanzamento e Terminazione:** Il contatore avanza di un'unità (`IncrementBuffer`). Al raggiungimento della sesta cifra inserita, il ciclo principale si interrompe e la macro termina, passando il codice completo al sistema per la verifica.

WAIT HASH

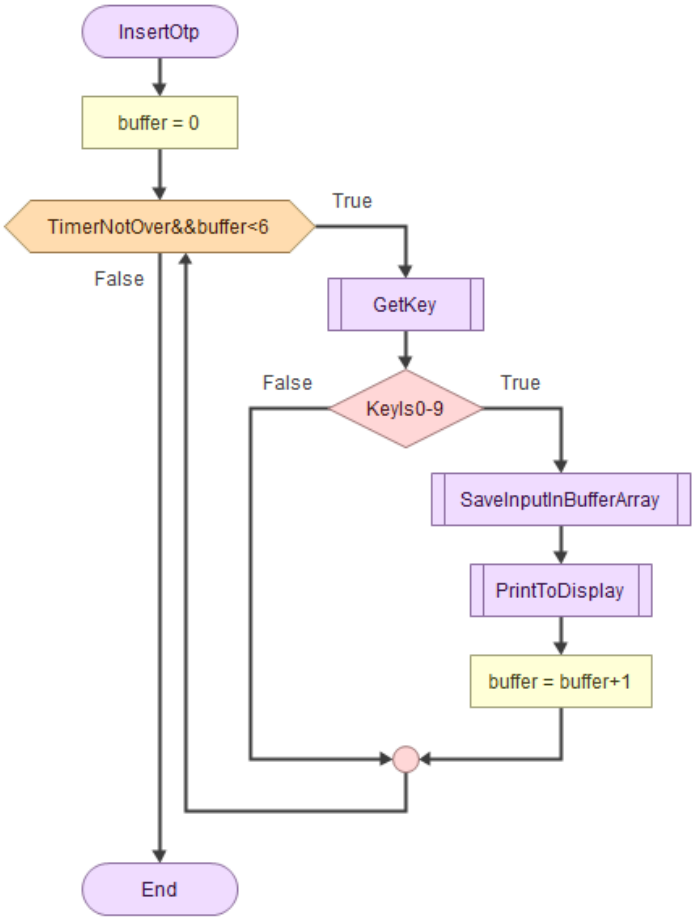
Il diagramma illustra la generazione del codice OTP e l'attesa della conferma utente, gestita tramite un ciclo temporizzato a (*Flag*):



- **Inizializzazione:** Il sistema genera e stampa l'OTP, avvia il timer hardware e azzera la variabile di controllo (`Flag = 0`).
- **Ciclo while:** Il flusso reitera la lettura della tastiera (`GetKey`) finché il timer è attivo e la Flag è falsa. Se viene rilevata la pressione del cancelletto (`#`), la `Flag` viene posta a 1, forzando l'uscita immediata dal ciclo.

  Alla fine della macro si valuta la causa dell'interruzione verificando lo stato del timer (`TimerNotOver`). Se il tempo è scaduto, il sistema devia verso lo stato di allarme (`Fail`). Se l'utente ha confermato in tempo il sistema consente di procedere all'inserimento del codice.

INSERT OTP



Il diagramma illustra l'acquisizione del codice OTP a 6 cifre sotto un rigido vincolo temporale, fondendo il monitoraggio del timer con il riempimento della memoria:

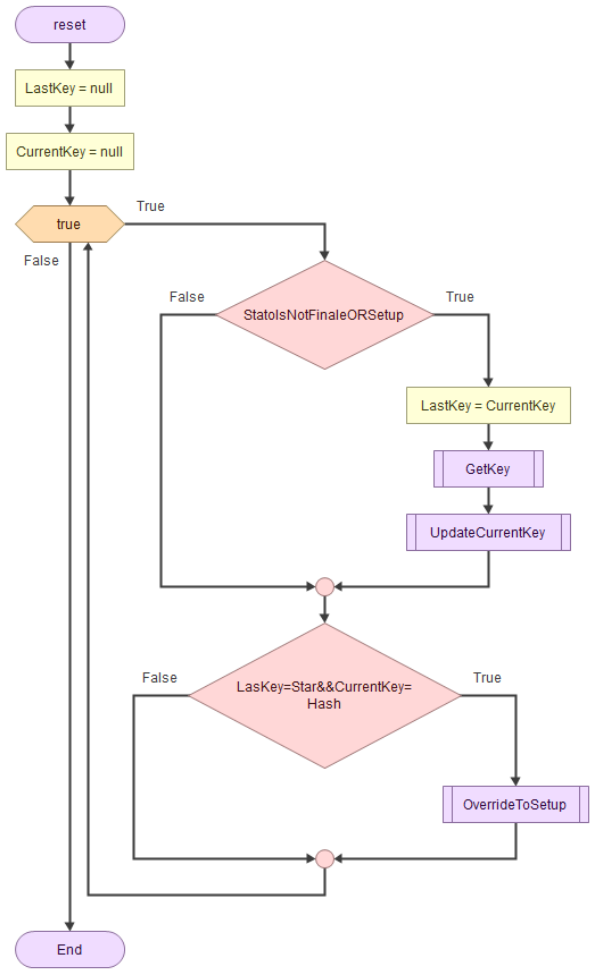
- **Inizializzazione:** Azzeramento del contatore delle cifre (`buffer = 0`).
- **Ciclo Ibrido (While):** Il sistema scansiona la tastiera finché il timer è attivo e il buffer non è pieno (`TimerNotOver && buffer<6`).
- **Filtro e Salvataggio:** Gli input non numerici vengono scartati a vuoto. Le cifre valide (0-9) vengono salvate in memoria, mostrate sul display a 7 segmenti e il contatore avanza.

  Alla fine della macro, si verifica lo stato del timer. Se il tempo è scaduto, il sistema va in blocco (`Fail`). Se il timer è ancora valido, significa che l'utente ha inserito tutte le cifre in tempo il sistema passa allo stato OPEN

Il cuore di questa macro riprende **esattamente la stessa logica di accumulo e filtraggio vista nella macro `InsertPin`**.
In entrambi i diagrammi:

1. **Il sistema fa da "filtro":** ignora i caratteri speciali e accetta solo input numerici.
2. **Riempimento sequenziale:** si utilizza un contatore per limitare l'inserimento a un massimo di 6 caratteri, avanzando solo quando l'input è valido.

OVERRIDE RESET



1. Inizializzazione del Modulo All'avvio o a seguito di un evento di `reset`, il sistema azzera la memoria di inserimento. Le variabili `LastKey` e `CurrentKey` vengono resettate. Questo garantisce che non vi siano letture residue nei buffer prima dell'inizio delle operazioni.

2. Ciclo di Scansione Il sistema entra in un ciclo di polling continuo while true, tipico dei sistemi *bare-metal* dove il task di scansione hardware viene eseguito ciclicamente

3. Controllo di stato Prima di interrogare fisicamente l'hardware della tastiera, l'algoritmo verifica lo stato logico del sistema tramite la condizione `StatoIsNotFinaleORSetup`.

- **Ramo True:** Se il sistema si trova in uno stato operativo standard (es. in attesa di star, inserisci pin...), la logica procede. Il valore corrente viene storicizzato (`LastKey = CurrentKey`), e si invoca la routine hardware `GetKey` per leggere la nuova pressione, seguita dall'aggiornamento del buffer con `UpdateCurrentKey`.
- **Ramo False:** Se il sistema è già in stato di Setup o in uno stato Finale (es. blocco per allarme o accesso già garantito), la lettura hardware viene bypassata. Questo ottimizza i cicli di clock della CPU, evitando di eseguire routine di I/O inutili quando l'input utente non è richiesto.

4. Rilevatore di Sequenza Il flusso converge nel blocco decisionale critico: `LastKey=Star && CurrentKey=Hash`. Questo blocco agisce come un riconoscitore di pattern a due stati.

- L'algoritmo valuta se l'utente ha premuto l'asterisco (`*`) seguito immediatamente dal cancelletto (`#`).
- **Ramo True:** Se la sequenza corrisponde, viene invocata l'azione `OverrideToSetup`. Questo trigger forza una transizione di stato asincrona, portando il sistema in modalità di Standby

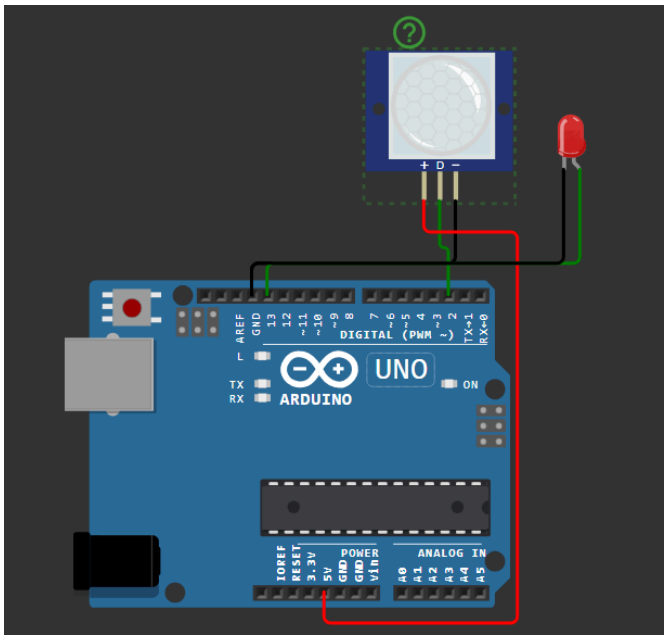
- **Ramo False:** Se la combinazione non è soddisfatta, il sistema ignora l'evento e conclude l'iterazione, ritornando all'inizio del ciclo per il campionamento successivo.

II. Sintesi

2.1 Modulo di Interrupt

link: [ESEMPIO INTERRUPT]

L'interrupt è costruito sulla combinazione di **configurazione booleana dei registri** (per definire la condizione di trigger), **mappatura statica della memoria** (per il posizionamento dei vettori di salto) e **priorità logica dell'hardware** (che sospende il Program Counter per forzare l'esecuzione della ISR).



2.2 Modulo Generatore OTP

1. Il Seed di entropia: Nel `main()`, la variabile `fast_entropy_counter` incrementa. Il valore che ha nell'istante esatto in cui l'utente preme l'ultimo tasto del PIN fa da *seed* (seme) iniziale, unico e imprevedibile per l'essere umano.

2. Algoritmo pseudo-casuale (Xorshift): Per calcolare le 6 cifre, il codice esegue un ciclo in cui rimescola i bit del seme usando l'algoritmo Xorshift a 16 bit:

Prende il numero, lo sposta a sinistra o a destra di *N* posizioni (`<<` o `>>`) e combina il risultato con il numero di partenza tramite l'operatore **XOR** (`^=`). Lo XOR genera un `1` solo se i bit calcolati sono diversi. Ripetendo questo "sposta-e-confronta" per tre volte con costanti matematiche precise (7, 9, 8), i bit si mescolano in modo caotico. Il numero finale sembra completamente casuale, ma è generato in pochissimi cicli di clock

3. Conversione ASCII: * Il codice isola gli ultimi 4 bit del numero rimescolato (ottenendo un valore tra 0 e 15).

- Se il risultato è maggiore di 9, sottrae 6 (riportandolo nel range 4-9).
- Converte il numero in char aggiungendo `'0'` e lo salva in `generated_otp[i]`

Valore Estratto (4-bit)	Condizione > 9	Sottrazione Applicata	Valore Decimale	Carattere ASCII (+ '0')
0	Falsa	Nessuna	0	'0' (48)
5	Falsa	Nessuna	5	'5' (53)
9	Falsa	Nessuna	9	'9' (57)
10	Vera	10 - 6	4	'4' (52)
15	Vera	15 - 6	9	'9' (57)

4. Invio seriale: L'array finale di 6 caratteri viene trasmesso bit a bit via UART per essere mostrato a schermo.

2.3 Modulo Schedulatore Temporale

Un Timer è un dispositivo che **conta il tempo**. Per fare ciò, il timer deve avere un evento di riferimento di base, chiamato T-base, di cui è noto il tempo trascorso. Nel nostro caso si tratta di un oscillatore a cristallo, che fornisce una **frequenza di clock pari a 16 MHz**,(Tb = 1/16µs).

Il timer incrementa il valore salvato (TCNT) in base a questo evento, il che significa che il tempo del timer è in funzione di T-base. Per ottenere un maggior controllo sul time step, ogni TIMER dispone di un **PRESCALER**, il quale moltiplica il numero di occorrenze dell'evento di riferimento necessarie per incrementare il valore effettivo del TCNT. Il tempo del timer può quindi essere calcolato utilizzando la seguente formula:

$$Time = Tb \times Prescaler \times TCNT$$

**L'unità di Time è quella di T-base (µs), essendo TimeValue solo un intero.*

Nel nostro caso utilizzeremo il **TIMER1** del nostro MCU, che è un timer a 16 bit, per avviare il countdown dell' OTP .

TCNT1, o Timer/Counter Register 1, è il registro principale associato al TIMER1. Esso tiene traccia del conteggio del timer, aumentando di uno ad ogni 1024 cicli di clock. Utilizzeremo questo timer per misurare con precisione il tempo tra gli impulsi e gestire le operazioni di temporizzazione necessarie per il nostro progetto. [1 in TCNT1 == 16µs realmente)]

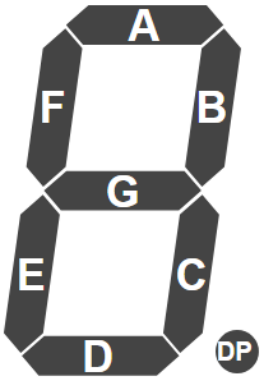
Il **TIMER2** è un timer a 8-bit utilizzato per gestire la visualizzazione dinamica sui display a 7 segmenti. A differenza del Timer1, questo lavora a una frequenza molto più alta per garantire che l'occhio umano non percepisca lo sfarfallio (*flicker*).

Utilizzando un prescaler diverso (impostato a 256), il timer genera un interrupt ogni 1 millisecondo

2.4 Modulo Visivo

link: [MODULO VISIVO TESTER]

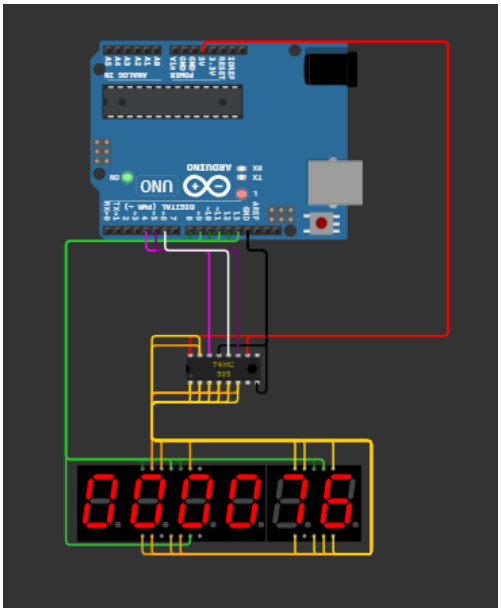
(aumenta una variabile e la mostra sul display)



Funzionamento del Display a 7 Segmenti

Mappatura e Codifica I segmenti sono standardizzati e identificati con le lettere dalla **A** alla **G** (come visibile nello schema). Per visualizzare un numero, il microcontrollore deve accendere una specifica combinazione di questi LED. Nel nostro progetto, lo stato di accensione è gestito tramite un singolo byte (8 bit), dove ogni bit corrisponde a un segmento secondo la mappatura convenzionale: `0bGFEDCBA`.

Ad esempio, per visualizzare il numero **"1"**, è necessario accendere solo i segmenti di destra, ovvero **B** e **C**. La codifica binaria corrispondente sarà quindi `0b00000110`
Logica di Pilotaggio Nel sistema sviluppato, il display è di tipo ad **Anodo Comune**. Questo significa che tutti i pin positivi (anodi) dei LED sono collegati insieme all'alimentazione. Di conseguenza, la logica di controllo è invertita ("attiva bassa"): per accendere un segmento, il pin corrispondente del microcontrollore deve essere portato a livello logico Basso (0V o `LOW`), mentre per spegnerlo deve essere portato a livello Alto (5V o `HIGH`). Il software gestisce questa inversione automaticamente applicando un operatore logico NOT (`~`) alla maschera binaria prima della trasmissione.



La logica del Multiplexing (Calcolo del tempo)

Per ottenere una frequenza di aggiornamento di 500Hz (ovvero 2ms per cifra) su 6 display, il Timer2 deve generare un interrupt ogni millisecondo.

Formula del Timer:

*f_interrupt = f_clk / (2 * N * (1 + OCR2A))*

Calcolo:

f_interrupt = 16.000.000 / (2 * 128 * (1 + 124)) f_interrupt = 16.000.000 / 32.000 f_interrupt = 500 Hz

- f_clk = 16.000.000 Hz
- N (Prescaler) = 128
- OCR2A = 124 (Il valore di comparazione)

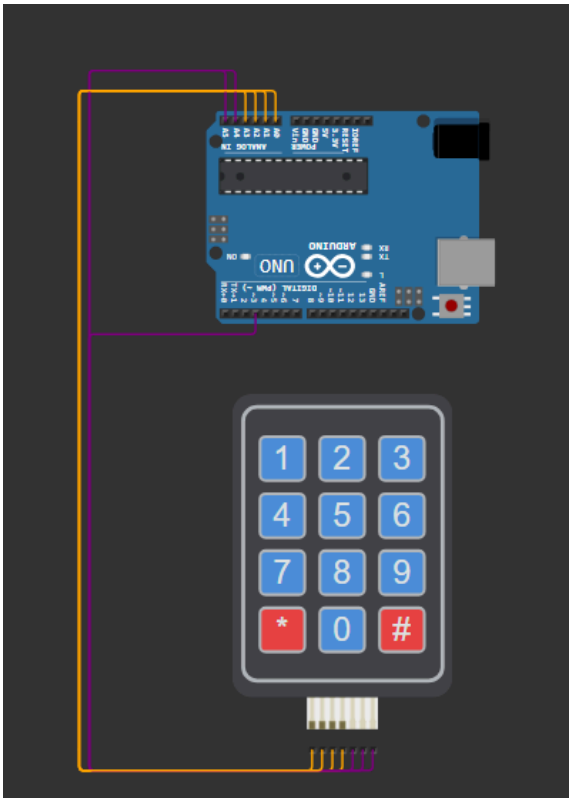
Uso dello Shift Register

Per pilotare il display a 6 cifre senza esaurire i pin del microcontrollore (sarebbero servite 14 connessioni), è stato utilizzato uno **Shift Register 74HC595**. Questo chip permette di controllare gli 8 segmenti usando **solo 3 pin** (Dati, Clock, Latch) grazie alla sua architettura a due stadi:

- **Stadio 1 - Flip-Flop (Scorrimento):** I dati vengono inviati dal microcontrollore in modo seriale (un bit alla volta). Ad ogni impulso di **Clock**, i bit scorrono in cascata all'interno degli 8 Flip-Flop fino a formare il byte completo.
- **Stadio 2 - Latch (Sincronizzazione):** Se il display fosse collegato direttamente ai Flip-Flop, i numeri sfarfallerebbero durante lo scorrimento dei bit. Il registro **Latch** risolve il problema funzionando come una barriera: mantiene le uscite "congelate" allo stato precedente e le aggiorna tutte simultaneamente (in parallelo) solo quando riceve l'impulso finale di Latch, garantendo un'immagine nitida e stabile.

2.5 Modulo Tastiera

link: MODULO TASTIERA TESTER



1. Scansione della Matrice (Hardware Polling)
La tastiera è una griglia di 4 righe (uscite) e 3 colonne (ingressi con resistenza di pull-up).

- Il microcontrollore abbassa a livello logico LOW una riga alla volta.
- Mentre una riga è LOW, legge le colonne: se il segnale di una colonna va al LOW, il tasto all'incrocio tra quella riga e quella colonna è premuto.
- Se non rileva nulla, restituisce un valore "vuoto" (`0xFF`).

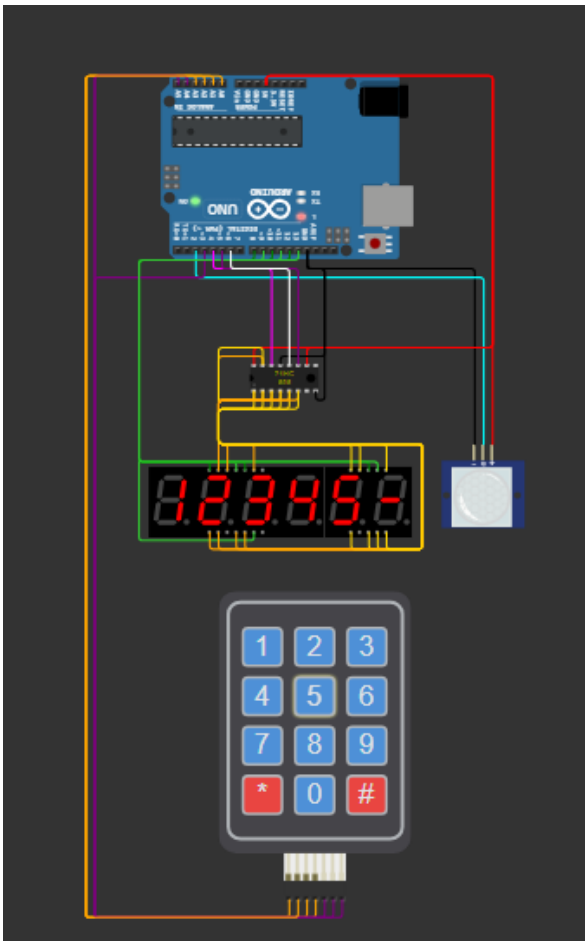
2. Macchina a Stati Finiti (Filtro Antirimbalzo)
I tasti fisici creano microscopici "rimbalzi" elettrici. Per evitare che un singolo tocco venga letto più volte, si usa una sequenza a 4 fasi:

- **Attesa Pressione:** Il sistema riposa finché non rileva un tocco.
- **Verifica Pressione:** Il tasto deve risultare premuto stabilmente per **16 cicli consecutivi**. Solo a questo punto viene considerato un input valido.
- **Attesa Rilascio:** Il sistema blocca nuove letture finché il dito non viene tolto dal tasto (evita le ripetizioni continue).
- **Verifica Rilascio:** Il tasto deve risultare completamente rilasciato per 16 cicli consecutivi prima di resettare il sistema e permettere una nuova digitazione.

3. Timer1 Per far sì che i "16 cicli" della macchina a stati corrispondano a un tempo reale e preciso, un timer hardware scatta esattamente ogni **1 millisecondo**. Il programma esegue una scansione e fa avanzare la logica solo quando scatta questo timer, garantendo un antirimbalzo perfetto di 16 millisecondi netti.

III. Sistema Complessivo

Finalmente abbiamo la comprensione necessaria per integrare tutti i componenti in e tradurre il flusso logico in codice C eseguibile. Per questo utilizzerò il simulatore WOKWI, ecco il PROGETTO



3.1 Manuale di Utilizzo

Per testare il sistema sul simulatore Wokwi, cliccare sul sensore PIR e attivare la simulazione di movimento per accendere il display. Il display mostrerà

`[ - - - - - - ]`.

Quindi cliccare il tasto * sul tastierino virtuale per avviare l'inserimento del PIN di sicurezza a sei cifre `[1 2 3 4 5 6]`;

se il codice è corretto, la centralina genererà un codice temporaneo (OTP) sul terminale, avviando un conto alla rovescia di 10 secondi, entro i quali sarà necessario cliccare # per confermare e ridigitare fedelmente l'OTP per validare l'accesso.

Se l'OTP inserito è corretto il display si trova nello stato `[1 1 1 1 1 1]`,

altrimenti `[0 0 0 0 0 0]`

Qualora si desideri annullare il test in qualsiasi momento, è sufficiente cliccare in sequenza la combinazione * e # per interrompere la procedura in corso, cancellare i dati in memoria e forzare il ritorno immediato del circuito allo stato di riposo.

DATA	DESCRIZIONE
2-4-2026	Prima versione analisi del problema
6-4-2026	Flowchart iniziale + Statechart
9-4-2026	Isolamento dei Moduli
11-4-2026	Realizzazione esempio Interrupt con Pir
13-4-2026	Inizio modulo visivo
20-4-2026	implementazione latch e flip flop d
25-4-2026	Implementazione modulo visivo
1-5-2026	Creazione timer 1s
2-5-2026	Revisione Flowchart iniziale
3–5-2026	Revisione statechart
10-5-2026	Implementazione modulo tastiera con filtro debounce
15-5-2025	Implementazione algoritmo xorshift per otp
17-5-2026	Sviluppo Flowchart logica
30-5-2026	Revisione Generale
7-6-2026	Flowchart logica di override
10-6-2026	Creazione Progetto modulare (Tastiera e display)
13-6-2026	Implementazione sensore pir per il risveglio sistema
20-6-2026	Fine sviluppo con aggiunta della logica di accesso e metodo di reset
30-6-2026	Revisione commenti codice